Nombres: Chávez Estefano, Magarisca Bryan, Tixicuro Daniel **Curso:** GR1CD

Ejercicio 2 - Analizador Léxico Básico con Lex/Flex

Preparación del Entorno y Estructura de Archivos

Para iniciar la práctica correspondiente al Ejercicio 2, lo primero que se realizó fue la preparación del entorno de trabajo en nuestro sistema operativo basado en Linux. Se trabajó dentro de un directorio específico designado para la materia, con el fin de mantener un orden adecuado. En este directorio, al finalizar toda la práctica, alojaremos todos los archivos necesarios para que nuestro programa funcione: el código fuente del analizador, los archivos de texto de prueba y los archivos generados automáticamente por los compiladores.

En un inicio, se realizó la preparación del entorno de trabajo en el sistema operativo basado en Linux, siendo entonces, que se trabajo en un directorio especifico designado para la materia, con el fin de mantener un orden secuencial y adecuado. En este directorio, al finalizar toda la práctica, se alojarán todos los archivos necesarios para el funcionamiento del programa, los cuales son los siguientes:

- Código fuente del analizador (`dos.l`)
- Archivos de texto de prueba (`entrada.txt` y `salida.txt`)
- Archivos generados automáticamente por los compiladores (`lex.yy.c` y `a.out`)

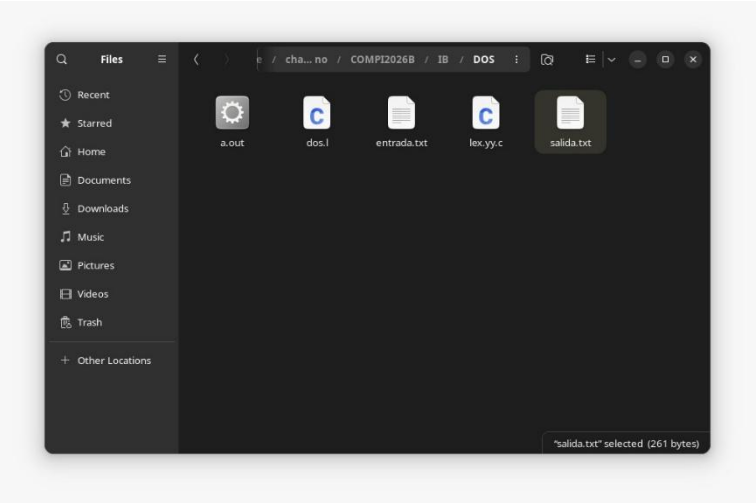


Figura 1 Contenido del directorio "DOS"

Desarrollo del Código Fuente (dos.l)

Para esta práctica, su núcleo principal se basó en un analizador léxico, y para ello, se utilizó un código fuente brindado por el docente, cuya extensión es `.l`, el cual va a tener el nombre de "`dos.l`". Este archivo se divide en tres secciones principales y fundamentales en la estructura Lex:

- **Sección de Declaraciones:** En este apartado se incluye la librería estandar de C, denominada como `<stdio.h>` para el posterior uso de la función `printf`
- **Sección de Reglas:** Se define lo que debe hacer el programa cuando encuentre ciertas palabras clave. Para este caso, la regla 'pare' ejecuta `printf('Comando pare recibido');` y la regla

‘inicio’ ejecuta `printf(‘Comando inicio ejecutado);`. Por otro lado, cualquier otro texto se copia a la salida estándar sin ninguna modificación.

- **Sección de Código C (Main):** Se realiza la llamada de `yylex()`, misma que inicia el ciclo de lectura y evaluación de reglas.

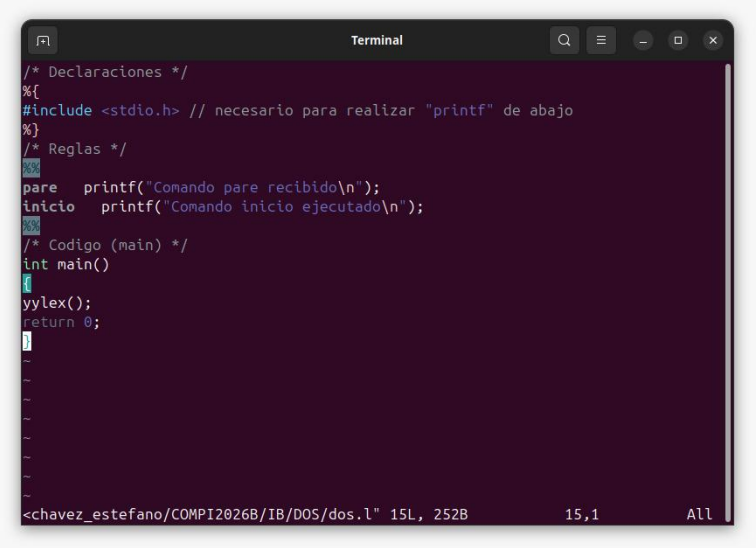


Figura 2 Código fuente de “dos.l”

Proceso de Compilación y Ejecución Interactiva

El proceso de la compilación consiste en dos pasos clave, siendo los siguientes:

1. **Generación del código en C:** Una vez que se ejecuta el comando ‘`lex dos.l`’, Lex lee las reglas del documento de texto y genera un archivo en C denominado como `lex.yy.c`
2. **Compilación del ejecutable:** Se utiliza GCC mediante el comando ‘`gcc lex.yy.c -lfl`’, en donde la bandera final `-lfl` es un elemento crítico para enlazar la librería de Flex, resultando de esta manera en el ejecutable `a.out`.
3. Una vez que se ejecuta el `a.out` mediante el comando ‘`./a.out`’, el programa detecta las palabras clave y responde respecto a las reglas preestablecidas, finalizando de esta manera el proceso de lectura mediante una señal de interrupción del programa activo

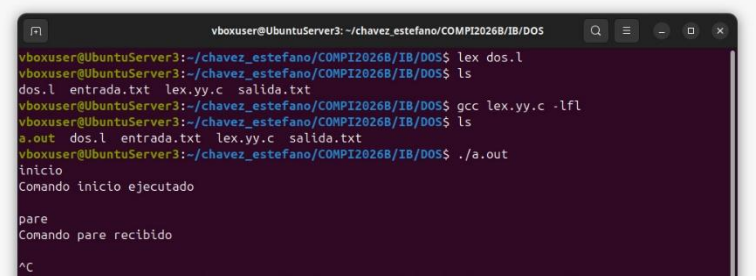


Figura 3 Proceso de compilación y ejecución del analizador de texto resultante

Pruebas con Archivos y Redirección de Flujos (I/O)

Para simular un ambiente más apegado a un entorno real, se probó el programa funcional, ingresando archivos de texto completo en lugar de tipear manualmente en la terminal.

- **Archivo de entrada:** Se redactó el archivo “`entrada.txt`”, mismo en el cual se entrelaza texto normal junto con las palabras clave para verificar la discriminación del analizador.

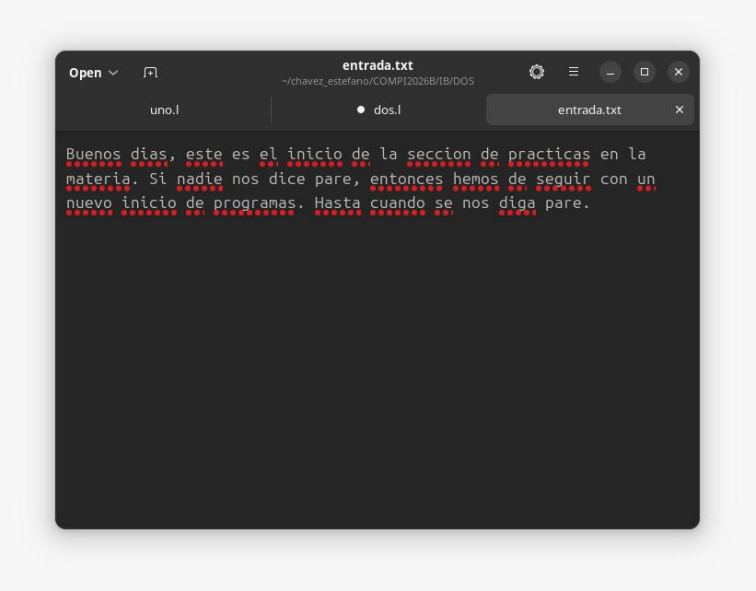


Figura 4 Contenido correspondiente al archivo de texto “entrada.txt”

- **Redirección de entrada y salida:** Se ejecutó el comando ‘`./a.out < entrada.txt > salida.txt`’, donde el símbolo < inyecta el archivo de texto previamente creado como una entrada estándar, mientras que el símbolo > captura la salida estándar del programa y la guarda en un archivo de texto nuevo llamado “`salida.txt`”

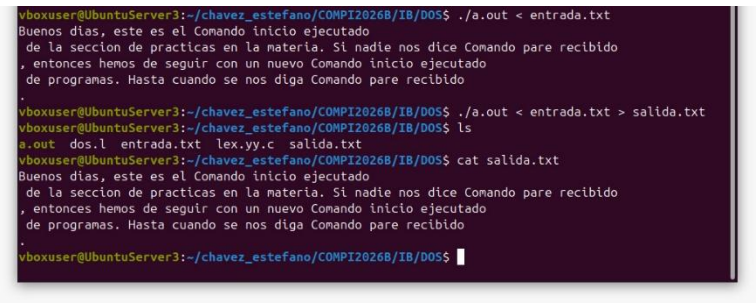


Figura 5 Ejecución de “a.out” con redireccionamiento de entrada y salida